Code Review

Code Quality
Readability:
The code only contains docstrings describing what each function does so readability could be improved with more useful comments

Maintainability:
The code utilizes modular programming which allows for easy maintainability

Usability Considerations
Graphical user interface implementation is observed which allows for a better user experience

Fault Tolerance
The system accounts for potential errors that may occur when unwanted inputs are given.

Code improvements
1: Under learner.py - line 153 and learning_module.py - line 135. Both lines have defined a method which has the same name “view_module()”. This could cause confusion in future code reviews/changes. It is suggested to change one of the methods’ name for better code clarity.

2: Feedback file is named as ‘feedpack.py’, this should be corrected as it could lead to confusion

3: Too many functions are defined in main.py. Some functions could be split to different files such as toggling between light and dark themes to allow for better maintainability.

Code - Requirements Alignment

3.1.1
“Distinctive & Secure Onboarding”
(In main.py, register() method - line 77)

“Distinctive & Secure Onboarding”
(In main.py, validate_password() method - line 426)
Enforcement of alphanumeric password not yet implemented

Fulfils Acceptance Criteria 5.1:

1. Users can register with unique usernames that meet the defined security criteria.
2. CodeVenture should prevent duplicate usernames from performing account registration.
3. Passwords must meet the specified complexity criteria (Only the specified length part not the alphanumeric part).

3.1.2
Password recovery not yet implemented

Acceptance Criteria 5.2 not fulfilled:

1. Users can initiate the password recovery process.
2. An email containing a time limit verification link for password reset is sent upon password recovery request.

3.1.3
"Transparent User Data Management"
(In user.py, edit_profile() method - line 90)
However, timestamp for profile update not yet implemented

Acceptance Criteria 5.3 fulfilled:

1. Users can modify their personal details through an intuitive interface

Acceptance Criteria 5.3 not fulfilled:

1. CodeVenture displays the timestamp of the last update for each user profile modification.

3.2.1
Content categorisation not yet implemented

3.2.2
Interactive learning not yet implemented

3.2.3
Real-time feedback not yet implemented

3.3
"Post-module quiz"
(In learner.py, view_learning_modules() method - line 131)
(In "self.modules[module_choice - 1].view_module() [Method of LearningModule class in learning_module.py] - line 147)

"Retry quizzes"
attempt_quiz() used in view_module() [Method of LearningModule class]
(In learner.py, view_learning_modules() method - line 131)

Suggestion:
There are two method with the same name, view_module() [Method of Learner class: in learner.py - line 153] and view_module() [Method of LearningModule class: in learning_module.py - line 135)

This causes confusion in future coding, a suggestion is to change one of the two methods to a different name such as view_selected_module().

"Hints & Solutions"
(In learning_module.py, attempt_quiz() - line 51)

3.4
“Visual Learning Journey Overview”
(In learner.py, view_achievements() method - line 193)

“Guided Learning Recommendations”
Recommendation system not yet implemented

Acceptance Criteria 5.5 fulfilled:

1. A progress bar or equivalent is displayed for each user.

Acceptance Criteria 5.5 not fulfilled:

1. CodeVenture recommends the next module or topic based on user progress.

3.5
“Centralised Learning Dashboard”
(In learner.py, dashboard() method - line 65

“Adaptive Cross-device Experience”
Cross-device not yet implemented

3.6
“Educator’s Interface”
(In educator.py, dashboard() method - line 35)
However, the implementation of educators’ ability to view students’ progress is not yet implemented.

“Administrative Console”
(In admin.py, all methods under line 7)

Acceptance Criteria 5.7 fulfilled:

1. Admins have a control panel for managing user accounts and content.

Acceptance Criteria 5.7 not fulfilled:

1. Educators have access to a view that tracks collective student progress.

3.7
“Inclusive Design”
Not yet implemented

“Data Privacy Compliance”
Not yet implemented

3.8
“User-driven Improvements”
Although feedback.py is made, no implementation
aligns with this requirement.

“Self-help resources”

(In user.py, view_faq() method - line 115)

3.9
“Operational Consistency”
TBD

“Load Resilience”
TBD

3.10
“Dynamic Content Adaptability”
Not yet implemented although some of the mentioned functions such as introduce/modify learning modules seem to be implemented in educator.py

Acceptance Criteria 5.11 not fulfilled:

1. Admins can add, edit, or remove modules

“Data Preservation”
Not yet implemented

3.11
“Collaborative Learning Environment”
Not yet implemented

“Performance Motivators”
Not yet implemented

“User-generated Challenges”
Not yet implemented

Acceptance Criteria 5.12 not fulfilled:

1. Users can participate in discussions in a community forum
2. Top-performing learners will be showcased on a leaderboard.
3. Users can create and solve problems in the community problems section.

4.1
“Performance”
The system's response time has been thoroughly tested under normal load conditions, and it consistently meets the requirement of not exceeding 2 seconds. As a result, users can expect a smooth and responsive experience when using the system.

4.2
“Usability”
This requirement is not met as while the code provides a graphical user interface, it doesn't explicitly address accessibility guidelines for users with disabilities. Ensuring that the GUI is accessible to everyone is an important consideration.

Therefore, Acceptance Criteria 5.8 is not fulfilled.

4.3
“Reliability”
Reliability cannot be guaranteed as while using Firebase for some functionalities does improve the reliability of this application, it depends on more factors such as on how the system architecture is implemented, which cannot be seen from the code base.

Therefore, Acceptance Criteria 5.9 is not fulfilled.

4.4
“Security”
The code does not meet the requirement that passwords should be hashed. In the code, passwords are stored as plain text.

Therefore, Acceptance Criteria 5.13 is not fulfilled.

4.5
“Availability”
While Firebase has its uptime guarantees, the uptime requirement for this application cannot be guaranteed as the app’s architecture is yet to be implemented fully.

Therefore, Acceptance Criteria 5.9 is not fulfilled.

4.6
“Maintainability”
The code is reasonably organised into classes and methods where each class has a clear purpose. Docstring explains functions and modular programming is utilised through the use of Object-Oriented programming principles. Variable and method names are descriptive. However, some parts of the code can be made more readable by adding comments, especially in complex validation functions.

4.7
“Scalability”
The code currently reads and writes user data to a text file "users.txt" (line 93 in main.py). While this approach may work for a small number of users, it is not suitable for handling a large number of users. Reading and writing to a text file could become very performance draining and lead to issues. To improve scalability, consider using a database like Firebase (which is already integrated) as Firebase is designed to handle large datasets.

4.8.1
“Cultural Sensitivity”
This requirement is fulfilled as the app’s content seems to be free from cultural biases, stereotypes and offensive elements and is solely focused on education.

4.8.2
“Advertising”
The system fulfils this requirement as there is indeed no advertisements to be loaded and displayed in this application.

4.8.3
“Handling of User Data”
The provided code does not explicitly address the requirement of not sharing user data with third parties without explicit consent. This requirement typically involves implementing data privacy and consent mechanisms, which are not evident in the code provided.

4.8.4
“Language”
Not yet implemented

4.8.5
“Feedback”
Not yet implemented

4.8.6
“UI”
The code provides a clean and friendly interface, suitable for young learners as well as users that are not great with technology. Users are not likely to be overwhelmed.

4.8.7
“Auditing and Standards”
There is not enough information to confirm whether regular audits are conducted to ensure the platform's ethical and inclusivity benchmarks.

4.9
“Compatibility”
The provided code, as it stands, does not fulfil the requirement of making the system accessible across various devices and platforms. It is primarily a desktop application developed using Tkinter and is not designed to be accessible on mobile devices or web platforms. To meet this requirement, significant modifications and additional development work would be necessary to extend the application's reach to different platforms. Therefore, Acceptance Criteria 5.14 is not fulfilled.

Code - Design Alignment

3. Class Diagram

- User:
  The user class in the class diagram states that userID will be inherited by its child classes Learner, Educator, Parent and Admin, as integer values. However, only Educator and Admin have unique IDs and the IDs are not of integer type.

  Class diagram shows that the User class will have method login(), logout() and updateProfile(). However, method login() and logout() has been implemented in main.py instead.

  updateProfile() is implemented but is named as edit_profile().

- Learner:
  progress attribute not yet implemented
  challenges attribute not yet implemented
  topics attribute not yet implemented

  startLearningModule() implemented but is named as view_learning_module()
  takeQuiz() is not implemented but instead in learning_module.py (under Quiz class named as attempt_quiz()).

- Educator:
  provideFeedback() not yet implemented

- Parents:
  viewChildProgress() not yet implemented,
  provideFeedbackToEdu implemented but now named as provide_feedback(). Also its only function only prints out the feedback content and does not actually send the feedback to the specified educator yet.

  setParentalControls() not yet implemented

- Feedback:
  feedbackID attribute not yet implemented

- ParentalControl:
  Class not yet implemented

- Admin:
  Method manageUser() is implemented but is broken into different methods:
  add_user() and remove_user.

- MessagingSys class not yet implemented

  generateReport() not yet implemented
  manageSysUpdate() not yet implemented
  viewSysLogs() not yet implemented

- LearningModule
  moduleID attribute not yet implemented
  content attribute implemented but named as description
  associatedQuiz attribute implemented but named as quizzes
  getContent() implemented but named as get_description()
  updateContent() implemented but named as edit_learning_module()

- Quiz:
  quizID attribute not yet implemented
  questions implemented but as String type instead of a list that contains String type objects
  associatedModule attribute not implemented

startQuiz method implemented but named as attempt_quiz()
gradeQuiz method not yet implemented

- Challenge:
  Class not yet implemented

- Achievement:
  Class not yet implemented
  It seems to be implemented as a string as evidenced in the implementation of “achievement: str = None” under LearningModule class (In learning_module.py - line 110)

- Topic
  Class not yet implemented

- SystemUpdate:
  Class not yet implemented

- SystemLog:
  Class not yet implemented

- Message:
  Class not yet implemented

- UserDatabase
  Implemented but not as a class but instead as a text file (txt). This is evidenced by the code “with open("./data/users.txt", "a") as file:” (line 489). The lines of code is writing new user information into the text file “users.txt”.

- ValidationSystem
  Class is implemented however, it is split into different classes: Login, Register

  Login & Register located in main.py - line 214 & 277 respectively

  Login:
  validate_login() method - line 261

  Register:
  validate_email() method - line 415
  validate_password() method - line 425
  validate_register() - line 436

Other aspects such as the use case and sequence diagrams have no major issues.